engineering proceedings

MDPI

Proceeding Paper

SQL and NoSQL Databases: A Comparative Study with Perspectives on IA-Based Migration Approach †

Soukayna Abdellaoui [1,*], Wafae Abbaoui [1], Loubna Meziane [1], Brahim El Bhiri [2] and Soumia Ziti [1]

[1] Intelligent Processing and Security of Systems, Faculty of Sciences, Mohammed V University in Rabat, Rabat 10000, Morocco; wafae_abbaoui@um5.ac.ma (W.A.); loubna_meziane2@um5.ac.ma (L.M.); soumia_ziti@um5.ac.ma (S.Z.)

[2] Centre d'Excellence en Innovation et Recherche Appliquee, Harmony Technology, Rabat 10100, Morocco; bra.elbhiri@gmail.com

* Correspondence: soukayna_abdellaoui@um5.ac.ma

† Presented at the 7th edition of the International Conference on Advanced Technologies for Humanity (ICATH 2025), Kenitra, Morocco, 9–11 July 2025.

Abstract

Traditional relational databases have been widely used for many years and have been the go-to choice for the industry. However, the emergence of NoSQL databases coincided with the increasing popularity of the Internet, social networks, and cloud computing. Data migration has become a crucial topic, due to the rapid growth of applications and the ever-expanding amount of data being collected. This has led to a shift from relational database management systems (RDBMS) to NoSQL databases, also known as not only SQL. This article compares the characteristics of both database architectures, SQL and NoSQL, focusing on aspects such as scalability and performance, flexibility, and security. Additionally, the role of AI in streamlining the migration process will be explored.

Keywords: SQL; NOSQL; ACID; BASE

Academic Editor: Ayman S. Mosallam

Published: 20 November 2025

Citation: Abdellaoui, S.; Abbaoui, W.; Meziane, L.; Bhiri, B.E.; Ziti, S. SQL and NoSQL Databases: A Comparative Study with Perspectives on IA-Based Migration Approach. *Eng. Proc.* **2025**, *112*, 72. https://doi.org/10.3390/engproc2025112072

1. Introduction

Modern relational database systems often struggle to handle large amounts of data efficiently, especially when it comes to quick, affordable analysis and processing. They are not always the best fit for big data needs [1]. One recent study [2] recognized several issues that arise with the application of relational databases: more precisely, in big data management. Among the major challenges is the restricted ability of the relational database to efficiently process and integrate enormous volumes of data with various types and structures. This is especially difficult when it comes to managing data that is generated with high velocity: for instance, real-time data feeds or rapidly changing datasets. The need for these databases to process both the variety and velocity of data can result in performance bottlenecking and difficulties in maintaining data consistency, and this is one of the biggest challenges of big data management.

Many organizations are considering moving their data from traditional relational databases to NoSQL solutions [3]. This shift is often driven by the increasing size of stored data and the limitations of existing applications in handling issues like scalability and high availability. As emphasized in recent studies, NoSQL databases offer the ability to scale out easily through horizontal expansion, making them an appealing choice for mod-ern data needs [4–6]. Because of this, converting current SQL databases into NoSQL systems is becoming an important and growing trend.

Moving data from a relational database to a NoSQL database can be quite challenging. As noted in [7], the process involves figuring out the right schema for the NoSQL system and smoothly integrating the migration of the relational schema into the new NoSQL structure. To understand the benefits and challenges of migration, it is vital to conduct an adequate process.

The strengths and limitations of each model with respect to the management of volumes of data, scalability, and flexibility would be shown in the comparison of the architectural frameworks of the SQL and NoSQL databases. In addition to that, to make the whole process more efficient, AI could play a crucial role in automating schema mapping and in adapting data to conform to the new NoSQL formats.

This paper is divided into the following sections: we compare and contrast SQL and NoSQL architecture to highlight their fundamental differences; we talk about the challenges involved in a transition from an SQL system to NoSQL; and we talk about the use of artificial intelligence in this transition.

2. Comparison of SQL and NoSQL Architectures

SQL is a database programming language that manages the data in a Relational Database Managing System (RDBMS). RDBMS employs a relational model of table relationships with foreign keys, primary keys, and indexes. Because of this, storing and retrieving data is faster compared to earlier navigational models. SQL was initially developed by IBM. SQL is an industry-standard language that is used to query and manipulate data. SQL can retrieve data from the database and execute queries against the database. SQL also acts on a huge quantity of records in a database by inserting, updating, and deleting records. SQL can create a new database, a new table in a database, and store procedures and views. Apart from this, SQL can grant permission for tables, procedures, and views [8].

2.1. SQL Database Architecture

When we talk about architecture in databases, there are two main types: physical and logical. Physical architecture is all about how data actually sits in your storage—think of things like pages, extents, database files, and transaction logs. Meanwhile, logical architecture is how data is organized and shown to users. This includes structures like tables, constraints, views, stored procedures, functions, triggers, and more. SQL helps manage many of these parts by handling transaction controls, maintaining data integrity, managing permissions, creating views, and working with embedded or energetic SQL [8].

Relational databases are not well-suited to handle these new requirements, due to several limitations:

- They cannot efficiently process unstructured and random data, as they require a predefined schema in advance.
- Achieving horizontal scalability (distributing the workload across multiple servers) is challenging for RDBMS, as they adhere to ACID properties (atomicity, consistency, isolation, durability).
- They do not natively support object-oriented data structures, which force complex queries to manipulate stored objects, along with their associated data [9].

ACID, which stands for atomicity, consistency, isolation, and durability, describes key properties mainly used in database transactions. A transaction is a set of related operations on data. These ACID properties help ensure that data remains accurate and reliable. Atomicity means that either all parts of a transaction are completed successfully, or none are, so the database stays consistent. Consistency ensures that after a transaction, the data still follows the rules and stays in a valid state. Isolation makes sure that one trans-action does not interfere with others running at the same time, keeping things orderly.

Durability guarantees that once a transaction is committed, its changes will stay even if the system crashes later, so nothing is lost [8].

While ACID compliance is essential for systems in banking, finance, and security, it can introduce significant overheads for applications requiring extensive data sharing, such as those used by Google and Amazon. Traditional relational databases (RDBMS) struggle to meet certain requirements, including:

- Distributed architecture.
- Scalability.
- Fine-grained control over performance.
- High availability.
- Low latency.
- Cost-effectiveness to address these challenges: the concept of NoSQL was introduced [10].

SQL-based systems are well-suited for a range of applications, from fast write-oriented transactions to scan-intensive analytical workloads. However, since SQL relies on a predefined schema, it requires prior knowledge of the data structure, which can be difficult to achieve in the context of big data. Additionally, SQL is not designed to efficiently process unpredictable or unstructured information. Consequently, NoSQL databases were developed to overcome these limitations [8].

2.2. NoSQL Databases Architecture

The most popular database management system was the Relational Database (RDBMS). However, non-relational, cloud, or "NoSQL" databases are increasingly popular as an alternative database management paradigm. As shown in Figure 1, NoSQL databases offer several advantages such as scalability and flexibility. NoSQL databases are becoming increasingly popular for real-time online apps and handling big data. Sometimes, they are also called "not only SQL" databases, because they can include query languages similar to SQL, making them flexible and powerful [11].

Most NoSQL databases are open source, but they exist in the four data models listed below. All of these types possess some particular characteristics, but the different information models cross-validate each other. There have been four NoSQL types taken into consideration by the specific data storage model evolution [12].

Key value model: The original NoSQL model is considered the most basic, storing data in a schema-less way by using values and indexed keys. A key value store can be compared to a relational database with just two columns: the key or attribute name and its corresponding value. This type of database schema is typically used for holding session data like shopping cart details, user preferences, and user profiles. As an illustration, let us consider a bank database, depicted in Figure 2.

Wide column store: stores information in big table-style databases as pieces of columns. Because it keeps data tables in columns rather than rows, it seems to be an inverted table and offers the benefits of maximum scalability and performance. For content management systems, blog platforms, keeping counters, heavy writing loads, and many more applications, the column database type is recommended. Figure 3 below shows the broad column data store's structure.

Document databases: Document stores are a more advanced type of key value store, where values are stored as documents in complex structures. The document is at the core of this NoSQL database model, providing a simple way to store and manage document-oriented data in formats such as XML, JSON, BSON, and others. These databases are frequently utilized in e-commerce, blogging platforms, content management systems, and web analytics. Figure 4 below depicts the structure of document store databases, displaying various databases with unique IDs (A, B, C, D), containing interconnected information.

Volume	• Supports large volumes of stored data—terabytes to exabytes.
Velocity	• Designed for high-speed processing of data—ideal for streaming data with response times ranging from milliseconds to seconds.
Variety	• Supports different types of data—structured, unstructured, and text data.
Veracity	• Supports uncertain or inconsistent data—due to latency, ambiguity, or wrong information.
Non-relational Model	• Not based on the traditional tables and not reliant on SQL to manipulate data.
Flexible Schema	• Uses different types of data models, like key-value stores, document databases, col-umn-family models, and graph databases.
Relational Alternative	• A replacement for classical relational database systems.
Handling Unstructured Data	• Optimized to manage unordered, irregular, and random data in an efficient manner.
Scalability	• Suited for managing large quantities of distributed data.

Figure 1. Advantages of using NoSQL databases [13].

BANK DATABASE	
Key	Value
1	ID:1 Joining Date: 15-July-1985 Designation: Cashier
2	ID:2 Joining Date: 19-March-1982 Designation: Manager
3	ID:3 Joining Date: 4-April-1988 Designation: Front Desk Officer

Figure 2. Key value databases.

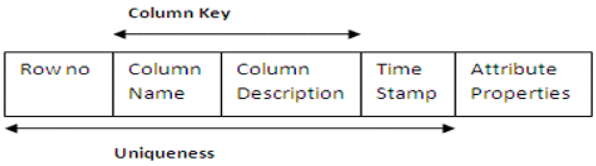

Figure 3. Structure of wide column data stores.

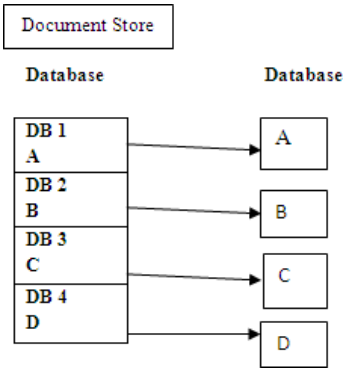

Figure 4. Structure of document store database.

Graph databases: This model, based on graph theory, becomes more complex when the data are interconnected and represented as a graph. The NoSQL database model allows for storing entities and relationships, with nodes representing entities and possessing properties. Data are stored only once in the graph structure and can be queried in various ways, based on relationships. This model is particularly useful for social networks, route information, spatial data, and approval engines. The structure of graph databases is shown in Figure 5.

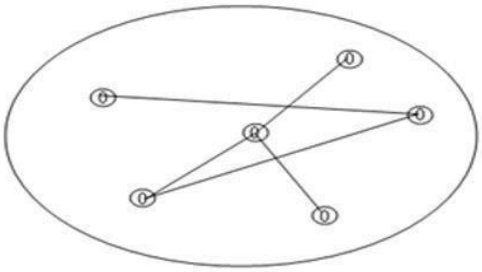

Figure 5. Structure of graph database.

3. Comparing SQL and NoSQL Database Features

Here, we will contrast and compare the characteristics of SQL and NoSQL databases like scalability, performance, flexibility, query language, security, data management, storage, and data availability. We will also contrast the pros and cons of both types of databases. Table 1 has a comparison of these characteristics in SQL and NoSQL databases.

Table 1. Critical differences between SQL and NOSQL databases models [14].

Properties	Type of Database	
	SQL	**NoSQL**
Structure of database	- Table-based databases	- Key value store databases - Wide column databases - Document store databases - Graph database
Flexibility of schema	- Predefined schema	- Dynamic schema
Guaranteed properties	- Atomicity, consistency, isolation, and durability (ACID principles)	- Basically available, soft state, eventual consistency (BASE properties), in compliance with CAP *
Security	- Robust security due to structured data, authentication, and access control means.	- More exposed due to distributed data, lack of encryption, and weaker authentication methods.
Data management (storage and access)	- Makes use of normalization to reduce redundancy and maximize storage.	- Utilizes denormalized storage, which may be redundant.

Table 1. *Cont.*

Properties	Type of Database	
	SQL	NoSQL
Use cases	- Suitable for transactional systems and banking.	- Best for massive data, real time, social media, and IoT.

* CAP theorem states the limitation of all databases with regard to the capacity to select only two among the three properties known as CAP (C—consistency, A—availability, and P—partition tolerance).

Scalability and performance: Scalability is essential when choosing a database; unlike SQL's costly vertical scaling, NoSQL offers cost-effective horizontal scaling with automatic data partitioning for better performance and data distribution [15,16]. SQL databases follow ACID for reliability and consistency, while NoSQL uses BASE for flexibility and availability, with the CAP theorem guiding trade-offs based on system needs [17,18].

Flexibility: NoSQL databases offer dynamic schema flexibility, making them ideal for evolving applications, while SQL databases require a fixed schema, making structural changes more complex and risky [19].

Query language: Relational databases use the powerful and standardized SQL, while NoSQL lacks a universal query language, making advanced queries and cross-platform use more challenging [20].

Security: Security is vital in DBMSs; while relational databases offer strong built-in protections, NoSQL databases often lack native security features and may require third-party tools to handle authentication, access control, encryption, and auditing [15,16].

Data management—storage and access: Relational databases use normalization to avoid redundancy and ensure clean, efficient storage, while NoSQL databases allow redundant data to be available, using replication (master–slave or master–master) to enhance reliability and prevent data loss [20].

4. Challenges in Migrating from SQL to NoSQL

A key challenge in migrating from SQL to NoSQL is converting structured tables into flexible, schema-less formats, while also managing complex inter-table relationships without traditional joins or foreign keys. This complexity is further compounded by the scale of modern databases, where manual data conversion becomes impractical. Poor migration efforts can lead to data loss, structural inconsistencies, and performance issues, making careful planning, automation, and thorough testing essential for a successful NoSQL migration [9].

Now there are numerous relational and NoSQL databases. This study relies on an ex-ample of MySQL to CouchDB migration described in an article to illustrate issues in the process of transition. However, porting an RDBMS to a NoSQL database like CouchDB is a big job, due to the fundamental differences between their data model and storage scheme. In an RDBMS, data are stored in normalized tables with strict relationships, whereas CouchDB is denormalized and loose in its model. The main job is therefore to convert relational tables into NoSQL documents while ensuring data consistency and making them capable of performing the same operations earlier.

CouchDB's key value approach allows for replication of certain relational database internal components, providing greater flexibility and preventing the need to update the schema whenever there is a change in requirements. However, unnormalized data conversion to semi-structured or unstructured data means an exact manipulation of relation-

ships and dependencies between the data. The complexity is seen in Figure 6, which shows the different steps and constraints of this migration [15].

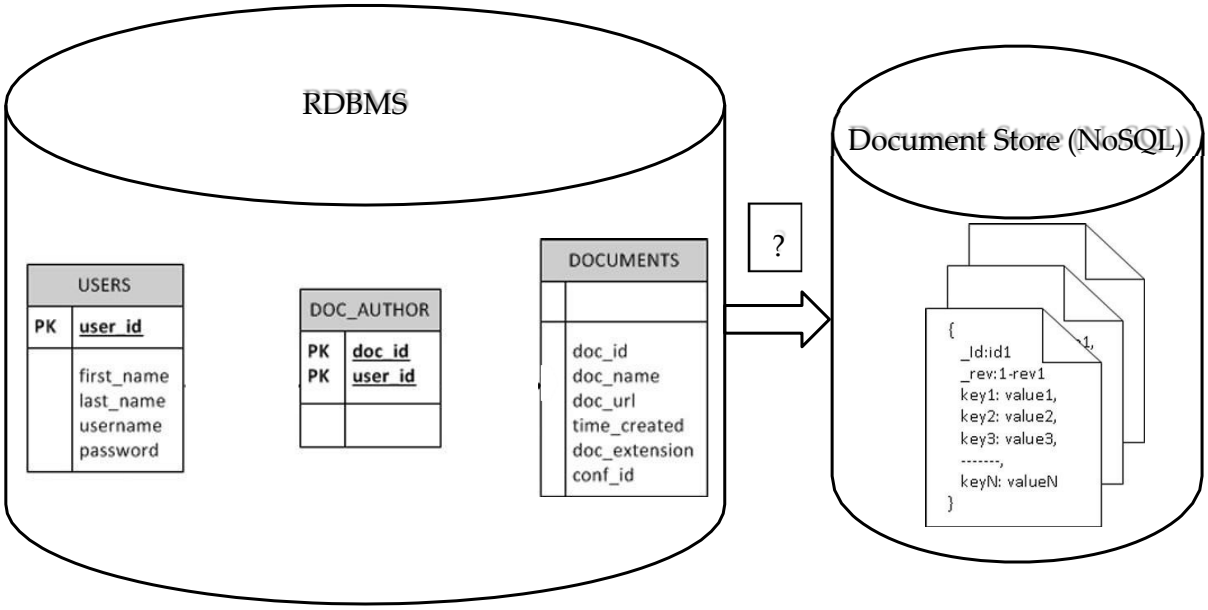

Figure 6. How data migrate from RDBMS to NoSQL [21].

5. The Role of Artificial Intelligence in SQL to NoSQL Migration

Artificial intelligence (AI) is the solution to simplifying, accelerating, and optimizing migration from relational databases (SQL) to non-relational databases (NoSQL). By leveraging its capabilities in analytics, intelligent modeling, and automation of complex operations, AI reduces technical barriers and operational inefficiency, and human intervention accelerates the transition and makes it more uniform.

Schema analysis and adaptive transformation migration of schema structures is one of the most crucial steps in database migration. SQL databases follow a rigid, predefined schema, while NoSQL databases follow flexible and dynamic schema structures. AI facilitates this by intelligently examining the SQL schema, extracting table relationships, and giving the optimal NoSQL schema structure. With machine learning algorithms, AI can dynamically determine the best NoSQL data model—document-based (e.g., MongoDB), column-family (e.g., Cassandra), key value (e.g., Redis), or graph-based (e.g., Neo4j)—based on application needs, query patterns, and data volume. It ensures data integrity with optimal performance and storage.

5.1. Automated Data Migration and Consistency Management

AI-enabled automation becomes vital in data extraction, transformation, and loading (ETL). Traditional migration methods take enormous amounts of human labor, which in-crease the likelihood of mistakes, discrepancies, and data loss. AI eliminates these risks by automatically detecting duplicate, inconsistent, or incomplete data in the SQL database and rewriting it into proper NoSQL format. AI also generates transformation scripts that handle complex data mapping, relation translation, and indexing to enable seamless and error-free migration. AI-based real-time validation systems also provide an additional layer of data consistency by ensuring data integrity and preventing corruption in distributed NoSQL systems.

5.2. Adaptive Optimization and Performance Tuning

After migration, AI continues to play a vital role in the performance tuning of NoSQL databases. It can monitor query execution time, detect hotspots, and provide recommendations on indexing schemes, caching, and partitioning strategies to maximize efficiency. AI-powered analytics continually learn from usage patterns in the database, automatically adjusting settings such as the replication settings, sharding strategies, and load balancing to optimize for scalability and resilience. Furthermore, AI assists with anomaly detection, proactively identifying performance issues and suggesting resolutions ahead of time, before they impact operations.

In addition to the initial migration, AI facilitates continuous database evolution and management. As application needs evolve and data increases, AI can forecast future scalability requirements and suggest proactive modifications to the NoSQL infrastructure. It can automate tasks like data archiving, backup scheduling, and security improvements, minimizing administrative burdens. Moreover, AI-based predictive analytics assist organizations in optimizing resource allocation and storage usage, promoting cost-effective- ness and high availability.

Artificial intelligence-driven automation and intelligence revolutionize SQL to NoSQL migration by making it quicker, accurate, and scalable. By automating data trans-formation, schema analysis, performance optimization, and continuous management of database operations, AI enables a seamless transition with minimal downtime. With constant advances in AI technology, database migration capabilities will only continue to grow, enabling firms to leverage the adaptability and scalability of NoSQL with guaran-tees of data integrity and business effectiveness.

6. Conclusions and Future Perspectives

Both SQL and NoSQL databases possess strengths and weaknesses. SQL databases offer high consistency, dependability, and integrity of data with adherence to ACID property, which is best suited for those applications where high accuracy is needed, such as banking. However, they are less flexible and scalable because of their structured schema. On the contrary, NoSQL databases are less rigid and scalable and can handle diverse data types as well as dynamic schemas and are therefore best applied to big data. Flexibility here is at the cost of hard consistency, but it has the benefit of faster performance and availability.

The SQL to NoSQL migration is a complex and error-prone process, due to differences in data models, relationships, and query languages among NoSQL databases. Unlike SQL's normalized language, the structure of NoSQL databases varies and requires drastic manual modifications to data models and application logic. AI solves this by automatically migrating this task. AI tools are capable of making SQL schema analysis, identifying patterns in data, and suggesting optimal NoSQL structures and even data conversion with minimal human involvement. This AI approach reduces errors, lowers the cost, speeds up migration, and makes NoSQL technologies more accessible for use across more organizations that deal with large volumes of data.

Author Contributions: Conceptualization, S.A., W.A. and L.M.; validation, S.A. and W.A.; methodology, S.A.; writing—original draft preparation, S.A.; writing—review and editing, S.A. and W.A.; supervision: S.Z., W.A. and B.E.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: No new data were created or analyzed in this study.

Conflicts of Interest: Brahim El Bhiri were employed by the Harmony Technology. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Hamouda, S.; Zainol, Z. Document-Oriented Data Schema for Relational Database Migration to NoSQL. In Proceedings of the 2017 International Conference on Big Data Innovations and Applications (Innovate-Data), Prague, Czech Republic, 21–23 August 2017; pp. 43–50. [CrossRef]
2. Assunção, M.D.; Calheiros, R.N.; Bianchi, S.; Netto, M.A.S.; Buyya, R. Big Data computing and clouds: Trends and future directions. *J. Parallel Distrib. Comput.* **2015**, *79–80*, 3–15. [CrossRef]
3. Gudivada, V.N.; Rao, D.; Raghavan, V.V. NoSQL Systems for Big Data Management. In Proceedings of the 2014 IEEE World Congress on Services, Anchorage, AK, USA, 27 June–2 July 2014; pp. 190–197. [CrossRef]
4. Schram, A.; Anderson, K.M. MySQL to NoSQL: Data modeling challenges in supporting scalability. In Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity, Tucson, AZ, USA, 19–26 October 2012; pp. 191–202. [CrossRef]
5. Guimaraes, V.; Hondo, F.; Almeida, R.; Vera, H.; Holanda, M.; Araujo, A.; Walter, M.E.; Lifschitz, S. A study of genomic data provenance in NoSQL document-oriented database systems. In Proceedings of the 2015 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Washington, DC, USA, 9–12 November 2015; pp. 1525–1531. [CrossRef]
6. Rajaram, K.; Sharma, P.; Selvakumar, S. DLoader: Migration of Data from SQL to NoSQL Databases. In *Cognitive Science and Technology, Proceedings of the International Conference on Cognitive and Intelligent Computing, Zhengzhou, China, 10–13 August 2023*; Kumar, A., Ghinea, G., Merugu, S., Hashimoto, T., Eds.; Springer Nature: Berlin/Heidelberg, Germany, 2023; pp. 193–204. [CrossRef]
7. Chickerur, S.; Goudar, A.; Kinnerkar, A. Comparison of Relational Database with Document-Oriented Database (MongoDB) for Big Data Applications. In Proceedings of the 2015 8th International Conference on Advanced Software Engineering & Its Applications (ASEA), Jeju, Republic of Korea, 25–28 November 2015; pp. 41–47. [CrossRef]
8. Binani, S.; Gutti, A.; Upadhyay, S. SQL vs. NoSQL vs. NewSQL—A Comparative Study. *Commun. Appl. Electron.* **2016**, *6*, 43–46. [CrossRef]
9. Mahmood, A.A. Automated Algorithm for Data Migration from Relational to NoSQL Databases. *Al-Nahrain J. Eng. Sci.* **2018**, *21*, 60. [CrossRef]
10. Ali, W.; Shafique, M.U.; Majeed, M.A.; Raza, A. Comparison between SQL and NoSQL Databases and Their Relationship with Big Data Analytics. *Asian J. Res. Comput. Sci.* **2019**, *4*, 1–10. [CrossRef]
11. Khasawneh, T.N.; AL-Sahlee, M.H.; Safia, A.A. SQL, NewSQL, and NOSQL Databases: A Comparative Survey. In Proceedings of the 2020 11th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 7–9 April 2020; pp. 013–021. [CrossRef]
12. Nayak, A.; Poriya, A.; Poojary, D. Type of NOSQL Databases and its Comparison with Relational Databases. *Int. J. Appl. Inf. Syst.* **2013**, *5*, 16–19.
13. Khan, W.; Kumar, T.; Zhang, C.; Raj, K.; Roy, A.M.; Luo, B. SQL and NoSQL Database Software Architecture Performance Analysis and Assessments—A Systematic Literature Review. *Big Data Cogn. Comput.* **2023**, *7*, 97. [CrossRef]
14. Babucea, A.-G. SQL OR NoSQL DATABASES? CRITICAL DIFFERENCES. *Ann.-Econ. Ser.* **2021**, *1*, 53–59.
15. Oussous, A.; Benjelloun, F.-Z.; Lahcen, A.A.; Belfkih, S. Comparison and Classification of NoSQL Databases for Big Data. In Proceedings of the International conference on Big Data, Cloud and Applications, Kenitra, Morocco, 4–5 April 2018.
16. Zahid, A.; Masood, R.; Shibli, M.A. Security of sharded NoSQL databases: A comparative analysis. In Proceedings of the 2014 Conference on Information Assurance and Cyber Security (CIACS), Rawalpindi, Pakistan, 12–13 June 2014; pp. 1–8. [CrossRef]
17. Plechawska-Wójcik, M.; Rykowski, D. Comparison of Relational, Document and Graph Databases in the Context of the Web Application Development. In *Information Systems Architecture and Technology: Proceedings of 36th International Conference on Information Systems Architecture and Technology—ISAT 2015—Part II*; Grzech, A., Borzemski, L., Świątek, J., Wilimowska, Z., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2016; Volume 430, pp. 3–13. [CrossRef]
18. Mohamed, M.A.; Altrafi, O.G.; Ismail, M.O. Relational vs. NoSQL Databases: A Survey. *Int. J. Comput. Inf. Technol.* **2014**, *3*, 598–601.
19. Patel, T.; Eltaieb, T. Relational Database vs NoSQL. *J. Multidiscip. Eng. Sci. Technol. (JMEST)* **2015**, *2*, 691–695.

20. Sahatqija, K.; Ajdari, J.; Zenuni, X.; Raufi, B.; Ismaili, F. Comparison between relational and NOSQL databases. In Proceedings of the 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, 21–25 May 2018; pp. 0216–0221. [CrossRef]
21. Mughees, M. Data Migration from Standard SQL to NOSQL. Master's Thesis, University of Saskatchewan Saskatoon, Saskatoon, SK, Canada, November 2013.